KATHMANDU UNIVERSITY

Department of Computer Science and Engineering

Dhulikhel, Kavre



A Project Report

on

"IoT Based Home Automation System"

[Code Number: ENGG 102]

(For the partial fulfillment of Year I/Semester II in Computer Engineering)

Built with an ESP32 microcontroller and a web-based dashboard

Submitted by,

Nishant Sah (041125-25)

Sandesh Shrestha (041141-25)

Dipra Shrestha (041136-25)

Samin Tharu (041155-25)

Grishma Thapa (038007-24)

Submitted to,

Mr. Santosh Shaha

Department of Computer Science and Engineering

July 27, 2026

CERTIFICATION



A PROJECT PROPOSAL
ON
IoT Based Home Automation System

by:

Nishant Sah (Roll No. 041125-25)
Sandesh Shrestha (Roll No. 041141-25)
Dipra Shrestha (Roll No. 041136-25)
Samin Tharu (Roll No. 041155-25)
Grishma Thapa (Roll No. 038007-24)

Approved by:

1. Project Supervisor

_____	Mr. Santosh shaha	_____
(Signature)	(Name)	(Date)

2. Head of the Department

_____	Mr. Pankaj Raj Dawadi	_____
(Signature)	(Name)	(Date)

Acknowledgement

We would like to express our sincere gratitude to everyone who supported us throughout the development of this project. First and foremost, we are thankful to our project supervisor, **Mr. Santosh Shaha**, for his continuous guidance, patience, and valuable feedback at every stage of this proposal, from finalizing the system architecture to reviewing the final report.

We would also like to thank the **Department of Computer Science and Engineering, Kathmandu University**, for providing us with the resources, lab access, and academic environment necessary to carry out this work.

We are grateful to our classmates and peers in the Kathmandu University Robotics Club for their helpful discussions and suggestions during the hardware assembly and testing phases of this project.

Finally, we would like to thank our families and friends for their constant encouragement and support throughout this project.

Nishant Shah
Sandesh Shrestha
Dipra Shrestha
Samin Tharu
Grishma Thapa

Abstract

Our vision of modern living has been transformed with the advent of the Internet of Things (IoT). Technology to manage home security, efficiency, and comfort has finally become readily available to improve our lives. However, with the bulk of homes still using traditional manually operated switches and wiring that may have become aged and inefficient, it has yet to impact the average household; old switches are unable to be controlled remotely, leading to inefficiency of space, and may struggle to keep up with the dynamics of modern life and demand. Affordable home automation solutions that turn average houses into "smart, connected homes" have thus been desperately needed.

To bridge this gap, we developed and built this complete IoT based home automation control system. Centered on theESP32microcontroller. Utilizing existing Wi-Fi in our homes, we connect commonly household appliances to our system which are controlled through simple web page reachable viaany smartphoneandany computer.

Fourrelaysare wired up to four household appliances and they are controllable by ESP32 on any desired status - on or off.

The system is alsoequippedwith aDHT22 temperature/humiditysensor to constantly monitor the current temperatureand humidity, allowing ESP32 to automatically control one fan appliance once there has been a significantchange to the threshold temperature and/or humidity.

The developed system will fulfill a scalable, economical, and high-performance goal by delivering an ease to the home management. This is all achieved with amodular and extendible nature, leaving it in good condition to be scaled as needed, we plan on implementing a voice-feed back systemusing a simpleMAX98357A and theINMP441input, it will allow system to announced what has happened. The project proves and shows how affordable theiot-implementationis can and could contribute to an improve quality of life and home-security; smart-home is finally achievable.

Keywords: Internet of Things (IoT), ESP32, home automation, DHT22, relay module, web dashboard, embedded systems, Wi-Fi control, voice feedback

Contents

Abstract **ii**

Abbreviations **vi**

1 Introduction **1**

1.1 Background 1
1.2 Objectives 1
1.3 Motivation and Significance 2
1.4 Expected Outcomes 2
1.5 Limitations 3

2 Related Works **4**

3 Procedure and Methods **6**

3.1 Development Process 6
3.2 Methodology and System Architecture 7
3.3 System Flow Diagram 8
3.4 Web Dashboard Interface 9
3.5 Testing of the Components 9

4 System Requirement Specifications **14**

4.0.1 Software Specifications 14
4.0.2 Entity Relationship (ER) Diagram 15
4.0.3 Hardware Specifications 15

5 Project Planning and Scheduling **18**

5.1 Tasks 18
5.2 Gantt Chart 20
5.3 Conclusion and Future Enhancements 21

References **23**

List of Figures

3.1 Entity-Relationship (ER) Diagram – Home Automation Data Model . . . 7
3.2 System Flow Diagram – User to Appliance Control 8
3.3 Smart Home Portal – ESP32 Web Dashboard UI 9
3.4 Wooden Enclosure Frame for the Home Automation Prototype (Pre-Paint) 11
3.5 Unit Testing – Relay Channel Driving a Light and a DC Fan 11
3.6 Sensor Testing – Live DHT22 Temperature and Humidity Readings on the Serial Monitor 12
3.7 Network Testing – ESP32 Connected to the Home Wi-Fi Network, Confirmed on the LCD and on a Smartphone 12
3.8 Dashboard Testing – Smart Home Portal Web UI Showing Live Sensor Data and Appliance Status 12
3.9 Integration Testing – Full System Running with Light, Fan, and Appliance Sockets Controlled Together 13

4.1 Hardware Circuit Diagram – ESP32 with 4-Channel Relay Module, DHT22 Sensor, and I2C LCD 17

5.1 Gantt Chart – Project Schedule (14 Weeks) 21

List of Tables

3.1 Development Process Phases 6

4.1 Software Components and Tools 14
4.2 Hardware Components and Specifications 16

5.1 Project Task List 18

Abbreviations

Abbreviation	Full Form
API	Application Programming Interface
CPU	Central Processing Unit
DC	Direct Current
ER	Entity-Relationship
ESP32	Espressif Systems' 32-bit Wi-Fi and Bluetooth microcontroller
GCC	GNU Compiler Collection
GPIO	General Purpose Input/Output
IDE	Integrated Development Environment
IoT	Internet of Things
IP	Internet Protocol
LAN	Local Area Network
MHz	Megahertz
I2S	Inter-IC Sound (digital audio interface used by the INMP441 and MAX98357A)
OS	Operating System
PC	Personal Computer
MEMS	Micro-Electro-Mechanical Systems (used in the INMP441 microphone)
RAM	Random Access Memory
SDK	Software Development Kit
TCP	Transmission Control Protocol
UI	User Interface
USB	Universal Serial Bus
Wi-Fi	Wireless Fidelity
DHT	Digital Humidity and Temperature sensor

Chapter 1: Introduction

This chapter introduces our smart home automation system, breaking down the background, core objectives, and the motivation behind the project. We will look at why a system like this is so necessary today, its real-world importance, and the practical outcomes we expect to achieve. Ultimately, this section highlights how the project addresses current gaps in the market and contributes to the growing field of IoT-driven home automation.

1.1 Background

The exponential growth of the Internet of Things (IoT) devices, has transformed the very interaction with our living spaces completely, for better. Todays IoT smart home gadgets simplify life, whilst saving us a good portion on electricity. With a low cost, affordable ESP32 microcontroller kit, offering a WiFi and Bluetooth chip in one, programmability has never been easier and connecting a device with the physical, and the online, through a smart device browser or even your PCs web browser has never become simpler.

The biggest current trend is the rise of the wireless, on-the-go, home automation gadgets, which have given rise to multiple households appliances and multiple switches controlled via few devices. Many commercially available devices require the use of a very niche third party, and the programming limitations can come between you creating the system the way you see it to. To make matters worse, even these complex, restricted systems are very expensive. Due to the many current limitations this means theres an ideal market to deliver the ultimate, simple and user friendly home automation system through using a 4 channel relay module alongside the ESP32 with a DHT22 temperature and humidity sensor to manage a fans automatically, all viewed through an individual, bespoke web system dashboard.

1.2 Objectives

- To design and develop a smart home automation system using the ESP32 microcontroller
- To control multiple electrical appliances using a 4-channel relay module through a web dashboard
- To use a DHT22 sensor to automatically switch a fan on or off based on measured temperature and humidity

- To enable remote monitoring and control of all relays through a custom web-based interface hosted on the ESP32

1.3 Motivation and Significance

The main idea of this project is to design a cost-effective and highly adaptable home automation system that is significantly more accessible and flexible than those currently available on the market. Most of the existing solutions have certain disadvantages, such as the extreme reliance on specific hardware or the use of proprietary software

applications. This project will address these issues, providing a completely independent and customizable platform for managing ones household appliances. To begin with, the project will utilize an ESP32 microcontroller to host the dashboard, thus eliminating the need for third-party applications. Furthermore, it will use a 4-channel relay module to allow the user to control multiple devices from a single dashboard. Finally, the system will be equipped with a DHT22 sensor to automate the fans operation based on the temperature and humidity in the room. The main purpose of this project is to create an easily customizable and affordable alternative to the existing commercial solutions. The proposed system will offer a variety of features, including

complete remote control of the four relays, automation of the fan operation, local management of WiFi devices, and status updates. As an additional feature, the system will provide voice feedback to the user in order to announce the changes.

1.4 Expected Outcomes

- A fully functional IoT home automation system built around the ESP32
- Ability to control four appliances remotely through a web dashboard hosted on the ESP32
- Automatic fan control based on live temperature and humidity data from the DHT22 sensor
- Reliable, real-time performance over the home Wi-Fi network
- A working breadboard prototype that lays the foundation for a future voice-feedback module built around an INMP441 microphone and a MAX98357A amplifier

1.5 Limitations

While the system meets its core objectives, the current prototype has a few limitations that are worth stating upfront:

- The dashboard and relay control depend entirely on the home Wi-Fi network; if the network goes down, remote/manual control from a browser is unavailable until connectivity is restored.
- The system is not yet integrated with any cloud service, so it cannot be controlled or monitored from outside the local Wi-Fi network (e.g., over mobile data).
- The automatic fan control uses a single fixed hysteresis threshold rather than a configurable schedule or multi-zone logic.
- The relay module switches AC loads up to its rated 10A/250V per channel; it is not designed for higher-power appliances without additional hardware.
- The voice-feedback module (INMP441/MAX98357A) is currently a planned enhancement and has not yet been integrated into the working prototype.
- The current build has been tested on a breadboard prototype only and has not been evaluated for long-term reliability in a permanent installation.

Chapter 2: Related Works

This section explores past research and key technological milestones in the field of IoT-based home automation, particularly systems built on the ESP32 platform.

Several recent projects have used the ESP32 microcontroller as the core of a smart home system because of its built-in Wi-Fi, low cost, and ease of programming. Anusha et al. (2024) presented an ESP32-based home automation system that combines a relay module with sensor monitoring and reported that the platform reliably switched multiple appliances over a home Wi-Fi network. Their work shows how a single low-cost board can take on both switching and sensing duties without needing separate hardware for each task.

Other researchers have paired the ESP32 with cloud services or mobile apps rather than a self-hosted dashboard. Kakani et al. (2024) built an ESP32 relay system controlled through the Blynk app, allowing appliances to be switched from a smartphone even when the home Wi-Fi network is briefly unavailable. While convenient, this kind of design keeps the user dependent on a third-party app and account, which is one of the gaps this project tries to close by hosting the dashboard directly on the ESP32 itself.

On the hardware side, low-cost digital sensors like the DHT22 have become a standard choice for hobbyist and academic IoT projects because they report both temperature and humidity over a single data pin with reasonable accuracy (Adafruit Industries, n.d.-a). In a similar way, I2S digital audio components such as the INMP441 microphone and the MAX98357A amplifier are increasingly used to add voice input and audio output to microcontroller projects, since both connect directly to a microcontroller's I2S pins without needing a separate sound card or DAC (Adafruit Industries, n.d.-b; InvenSense Inc., n.d.).

From a software perspective, the ESP32 can be programmed either with the low-level ESP-IDF framework or with the more beginner-friendly Arduino core for ESP32, both maintained by Espressif Systems (Espressif Systems, n.d.-a, n.d.-b). The Arduino core trades a small amount of performance for a much shorter development time, which makes it a practical choice for a project like this one that needs to combine Wi-Fi, a web server, sensor reading, and eventually I2S audio in a single firmware image.

Looking at the existing literature, most ESP32-based home automation projects fall into one of two groups: systems that depend on a third-party app or cloud platform for control, and systems that expose a web dashboard but stop at simple on/off switching

without reacting to sensor data on their own. There is comparatively little work that combines a fully self-hosted web dashboard, sensor-driven automatic control such as temperature-based fan switching, and spoken feedback in a single low-cost prototype. This project aims to bring these three pieces together using an ESP32, a 4-channel relay module, a DHT22 sensor, and a planned INMP441/MAX98357A voice-feedback module.

Chapter 3: Procedure and Methods

3.1 Development Process

Building this IoT-based home automation system requires a clear, step-by-step approach. By breaking the development down into structured, sequential phases, we can make sure every hardware and software component is thoroughly tested and integrated. Taking a systematic approach like this is the best way to catch errors early and ultimately deliver a reliable, highly efficient system.

#	Phase	Description
1	Requirements Analysis	Define the 4 appliances to control, sensor type (DHT22), dashboard layout requirements, and Wi-Fi network configuration
2	Hardware Selection	Procure ESP32 dev board, 4-channel relay module, DHT22 sensor, breadboard, jumper wires, 5V power supply, demo appliances
3	Circuit Design	Map ESP32 GPIO pins to relay IN1–IN4 and the DHT22 data pin on the breadboard; plan safe 5V power distribution for all components
4	Firmware: Web Server	Implement the ESP32 web server in Embedded C/C++: serve the dashboard page, and handle requests for relay toggling and sensor data
5	Firmware: GPIO & Relay Control	Write functions to set GPIO pins HIGH/LOW for each relay channel from the dashboard; test each channel individually with demo appliances
6	Firmware: Sensor Integration	Integrate DHT22 sensor reading; add threshold logic so the fan relay switches automatically when temperature/humidity cross set limits
7	Dashboard UI Design	Design the control dashboard in HTML and CSS, with buttons for all 4 relays and a live sensor readout; serve it from the ESP32
8	Integration & End-to-End Testing	Flash complete firmware; access the dashboard from a phone/PC browser; test all 4 relay toggles, automatic fan control, and sensor readings
9	Debugging & Optimization	Fix firmware bugs; verify stable Wi-Fi and dashboard response under continuous use
10	Voice Module Exploration (Future Enhancement)	Wire the INMP441 microphone and MAX98357A amplifier to the ESP32 I2S pins on the breadboard and test basic audio playback

Table 3.1: Development Process Phases

3.2 Methodology and System Architecture

At its core, this project relies on a straightforward IoT setup powered by the ESP32. The way it works is simple: you tap a switch on the custom web dashboard, and that command travels over your home Wi-Fi straight to the microcontroller.

As soon as the ESP32 gets the message, it works out which appliance is being controlled and toggles the right relay channel. Think of the 4-channel relay module as a set of smart, electronic light switches wired directly into your appliances – whether that's a lamp, a fan, or any other household device. In parallel, the ESP32 keeps reading the DHT22 sensor every few seconds, and if the temperature crosses a set threshold it automatically switches the fan relay on or off on its own, without waiting for anyone to tap the dashboard.

To keep the system responsive, we wrote the firmware in Embedded C/C++ using the Arduino core for ESP32. The code handles staying connected to Wi-Fi, serving the dashboard pages, reading the DHT22 sensor, and toggling the relay pins, all from a single loop running on the board. This keeps the whole setup lightweight, fast, and independent of any third-party cloud app – the dashboard lives entirely on the ESP32 and is reached simply by opening its IP address in a browser.

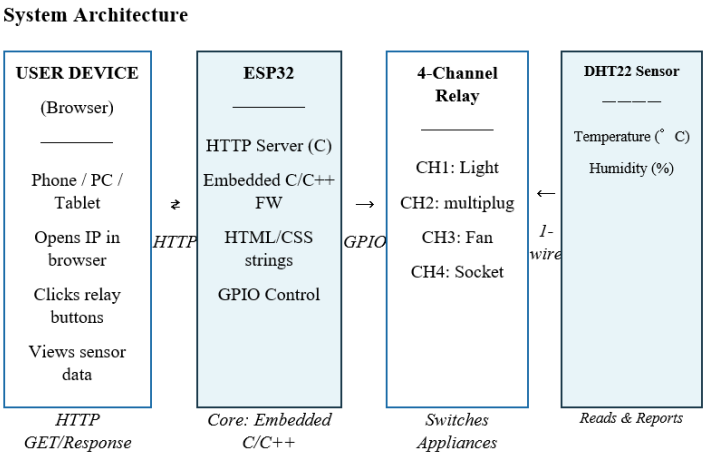


Figure 3.1: Entity-Relationship (ER) Diagram – Home Automation Data Model

3.3 System Flow Diagram

The flowchart below illustrates how a user command travels through the system to control an appliance, along with the parallel automatic path driven by the DHT22 sensor:

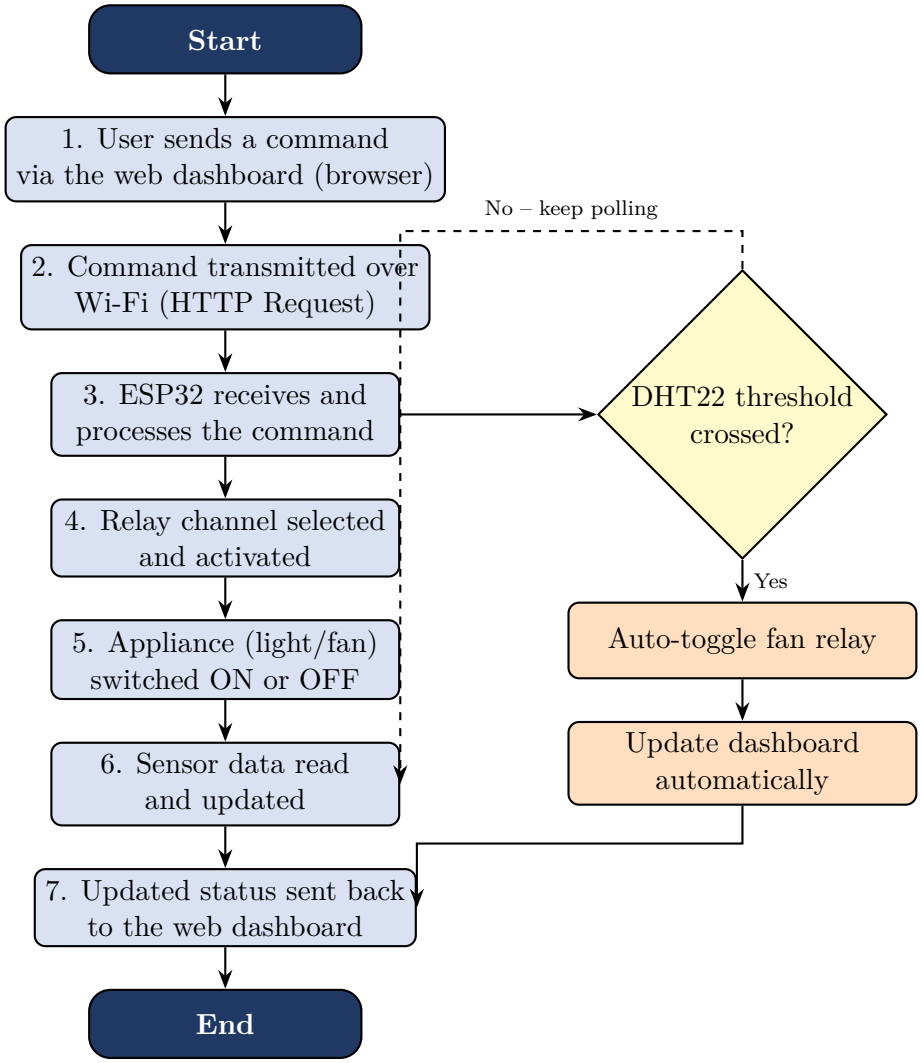


Figure 3.2: System Flow Diagram – User to Appliance Control

In parallel with the manual path (Steps 1–7), the ESP32 continuously polls the DHT22 sensor. Whenever the temperature or humidity crosses the configured threshold, it automatically toggles the fan relay and updates the dashboard, independent of any manual command from the user.

3.4 Web Dashboard Interface

The web dashboard is served directly by the ESP32 and is reached by opening the device's IP address in any browser on the same Wi-Fi network. It displays the live temperature and humidity readings from the DHT22 sensor, shows the current ON/OFF status of each of the four relay-controlled appliances with a toggle button for manual control, and lets the user set the automatic fan-control temperature threshold.

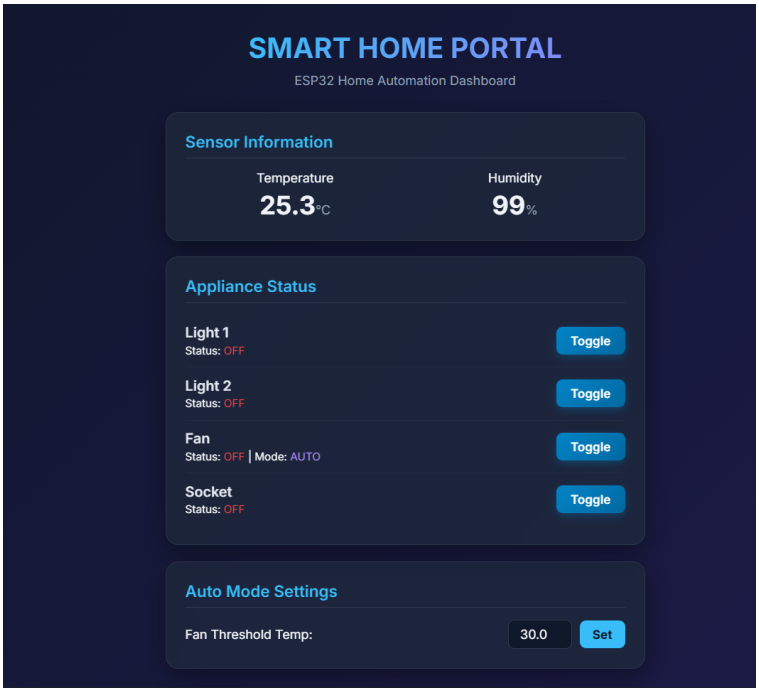


Figure 3.3: Smart Home Portal – ESP32 Web Dashboard UI

3.5 Testing of the Components

Before final integration, each hardware and software component was verified individually, and the assembled breadboard prototype was then tested end to end as a complete system.

Prototype Building

The breadboard prototype was built up incrementally rather than wiring everything at once. The ESP32 dev board was mounted first and its Wi-Fi connectivity was confirmed by flashing a minimal test sketch. The 4-channel relay module was then wired to GPIO pins D23, D22, D21, and D19, and each channel was driven HIGH/LOW individually to confirm correct switching before any appliance was connected. The DHT22 sensor was wired to a single data pin (D4) with its recommended pull-up resistor, and the 5V/GND rails were shared across the ESP32, relay module, and sensor using the breadboard's power rails to keep the wiring tidy and avoid ground-loop issues.



Figure 3.5 shows the assembled prototype running on the breadboard, with the LCD confirming that the ESP32 has successfully powered on and is connecting to the Wi-Fi network, while the relay module and DHT22 sensor are wired and ready to be driven by the firmware.

Physical Enclosure: Wood and Paint Work

Alongside the electronics, a wooden enclosure was built by hand to house the ESP32, relay module, and wiring in a safe, presentable case rather than leaving the prototype exposed on an open breadboard. The enclosure was cut and assembled from plywood into a compartmentalized frame, with separate internal sections to keep the mains-side relay wiring physically isolated from the low-voltage ESP32 and sensor wiring. Once assembled, the frame was sanded and finished with paint to give the final prototype a clean, safe, and professional appearance suitable for placement in an actual home.



Figure 3.4: Wooden Enclosure Frame for the Home Automation Prototype (Pre-Paint)

Testing

Testing was carried out in stages, moving from individual components up to the fully integrated system.

Unit Testing (Relay) Each relay channel was tested independently with a demo LED bulb and a small DC fan to confirm it switched ON/OFF correctly from a hardcoded firmware routine before the web dashboard was connected.

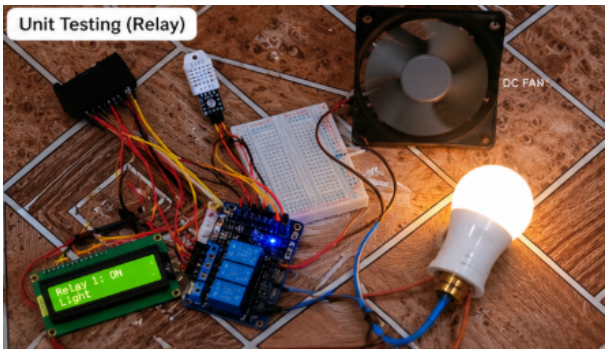


Figure 3.5: Unit Testing – Relay Channel Driving a Light and a DC Fan

Sensor Testing (DHT22) The DHT22 was verified by printing live temperature and humidity readings to the Arduino IDE serial monitor and comparing them against a reference thermometer.

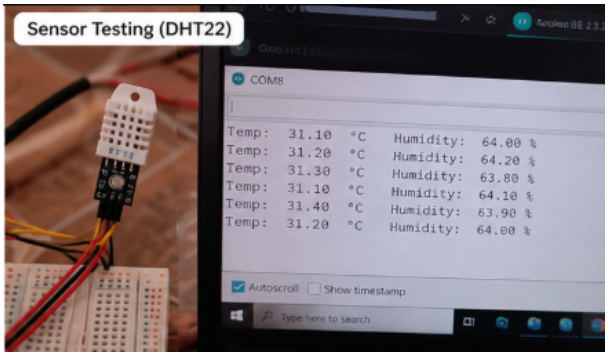


Figure 3.6: Sensor Testing – Live DHT22 Temperature and Humidity Readings on the Serial Monitor

Network Testing (ESP32 Wi-Fi) The ESP32's Wi-Fi connection was tested for stability by leaving it connected for extended periods and confirming it reconnected automatically after a simulated router restart. The LCD status display and a smartphone's Wi-Fi settings were both used to confirm a stable connection to the home network.



Figure 3.7: Network Testing – ESP32 Connected to the Home Wi-Fi Network, Confirmed on the LCD and on a Smartphone

Dashboard Testing (Web) The web dashboard was tested from multiple devices (phone and PC browsers) on the same network to confirm that relay toggle buttons, live sensor readouts, and the auto-mode threshold setting all worked as expected.

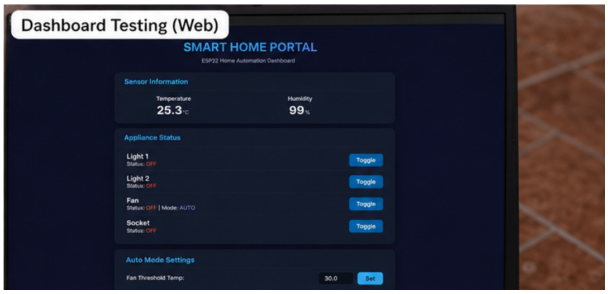


Figure 3.8: Dashboard Testing – Smart Home Portal Web UI Showing Live Sensor Data and Appliance Status

Integration Testing (Full System) Finally, the complete prototype was tested end to end – toggling each appliance from the dashboard, confirming the automatic fan control triggered correctly when the temperature crossed the set threshold, and checking that the dashboard reflected relay and sensor state changes in real time, with the light, fan, and plugged-in sockets all running together from the same system.



Figure 3.9: Integration Testing – Full System Running with Light, Fan, and Appliance Sockets Controlled Together

Each of these stages helped catch issues early – for example, incorrect GPIO pin mapping and an unstable Wi-Fi reconnect routine were both found and fixed during unit and network testing, before they could affect the integrated system.

Chapter 4: System Requirement Specifications

This chapter specifies the requirements of the proposed Home Automation System. It covers the software components and tools used to build the firmware and dashboard, the data model that connects the user, the ESP32, and the connected hardware, and the physical hardware components required to build the working prototype.

4.0.1 Software Specifications

The table below summarizes all software components, languages, and tools used in the system:

Component	Technology / Tool	Purpose
Microcontroller Firmware	Arduino Core for ESP32 (C/C++)	Wi-Fi initialization, web server, GPIO relay control, DHT22 sensor reading
Web UI – Structure	HTML (served by the ESP32 web server)	Dashboard layout: relay control buttons, sensor data display, status indicators
Web UI – Styling	CSS3 (linked/embedded in the dashboard page)	Button styling, colour-coded relay states (ON=green, OFF=red), responsive layout
HTTP Server	ESP32 WebServer / ESPAsyncWebServer library (C++)	Handles incoming HTTP requests from the dashboard; routes URLs to the correct relay/sensor handler
Sensor Library	Adafruit DHT sensor library (DHT22 driver)	Reads temperature and humidity from the DHT22 and drives the automatic fan-control logic
Firmware Compiler	Xtensa GCC toolchain (via Arduino/ESP-IDF)	Cross-compiles the C/C++ firmware for the ESP32 dual-core Xtensa LX6 CPU
Build System	Arduino IDE / PlatformIO	Manages compilation, library dependencies, and firmware upload
Firmware Flash Tool	esptool (command-line, open-source)	Uploads the compiled firmware binary to the ESP32 via USB serial
Code Editor / IDE	VS Code with PlatformIO, or Arduino IDE	Writing, editing, and organizing all C/C++ source files
Serial Debug Monitor	Arduino IDE Serial Monitor or PuTTY	Reads ESP32 debug output (IP address, sensor values, relay states)
Version Control		

Table 4.1: Software Components and Tools

4.0.2 Entity Relationship (ER) Diagram

The ER model below captures the key data entities in the Home Automation System and their relationships. The entities are: User, HTTP Request, ESP32, Relay Channel, Appliance, and Sensor Reading.

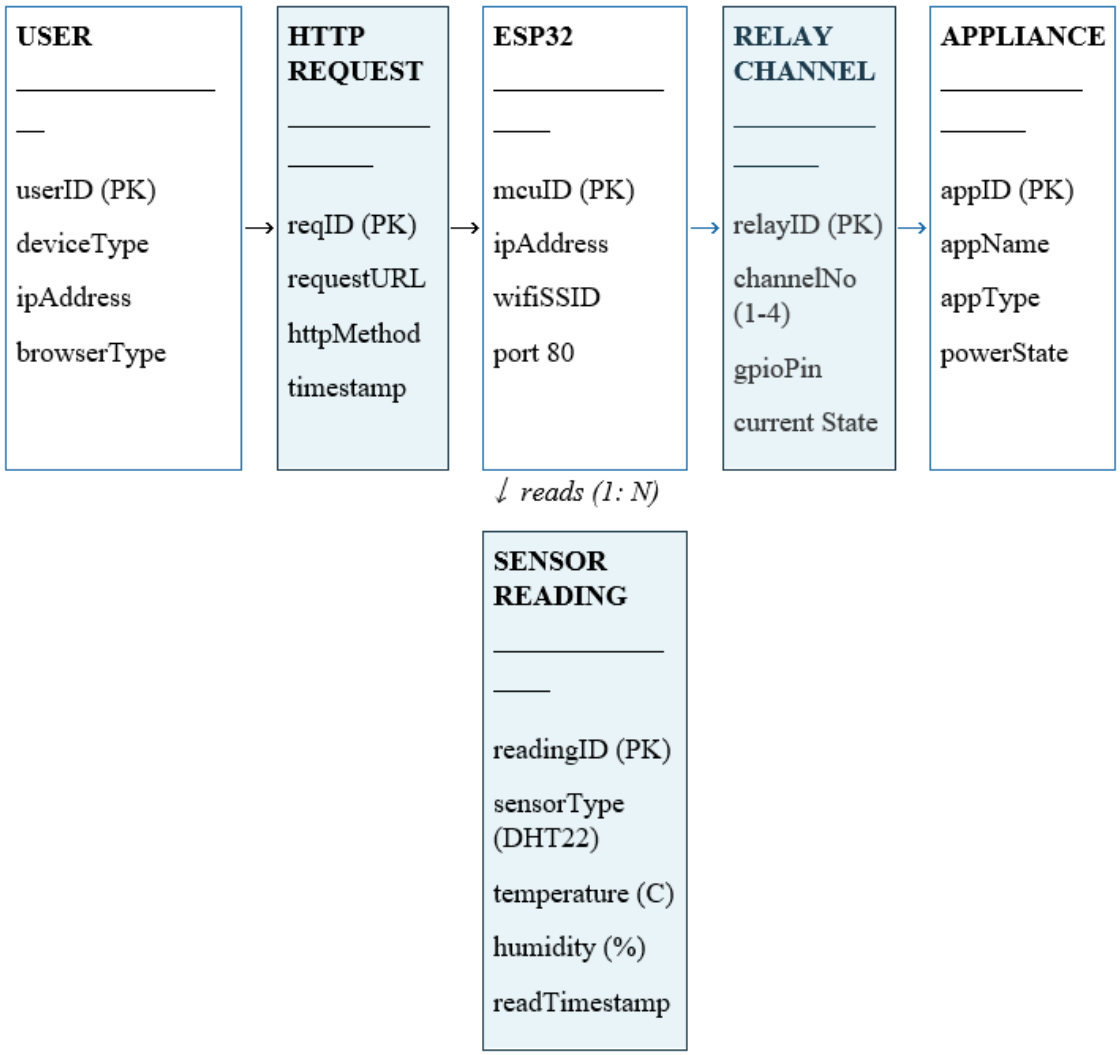


Figure 4.2: Entity-Relationship (ER) Diagram – Home Automation Data Model

4.0.3 Hardware Specifications

The table below lists all hardware components required for the system:

Component	Specification	Role in System
ESP32 Dev Board	Dual-core 240 MHz Xtensa CPU, 4 MB Flash, 802.11 b/g/n Wi-Fi + Bluetooth	Main microcontroller; runs the firmware, hosts the web dashboard, and reads the DHT22 sensor
4-Channel Relay Module	5V DC coil, 10A / 250V AC per channel	Electrically switches 4 home appliances ON or OFF
DHT22 Sensor	Temperature: −40 to 80 °C ±0.5 °C; Humidity: 0–100% RH ±2–5%; single-wire digital output	Measures ambient temperature and humidity and drives automatic fan control
Power Supply	5V / 2A DC adapter	Powers the ESP32 and relay module
USB Cable (Micro-USB/USB-C)	USB 2.0, data capable	Used to flash the compiled firmware onto the ESP32 from a PC
Breadboard & Connecting Wires	Standard breadboard, male-female jumper wires	Prototyping platform connecting ESP32 GPIO pins to the relay module and DHT22 sensor
Host PC / Laptop	Any PC running Windows, macOS, or Linux with Arduino IDE / PlatformIO	Compiles firmware and hosts the code editor
Wi-Fi Router	802.11 b/g/n, any standard router	Provides the local network that connects the ESP32 to phones/PCs running the dashboard
Demo Appliances	LED bulb (5V DC), small DC fan	Demonstrates relay switching and automatic fan control during testing

Table 4.2: Hardware Components and Specifications

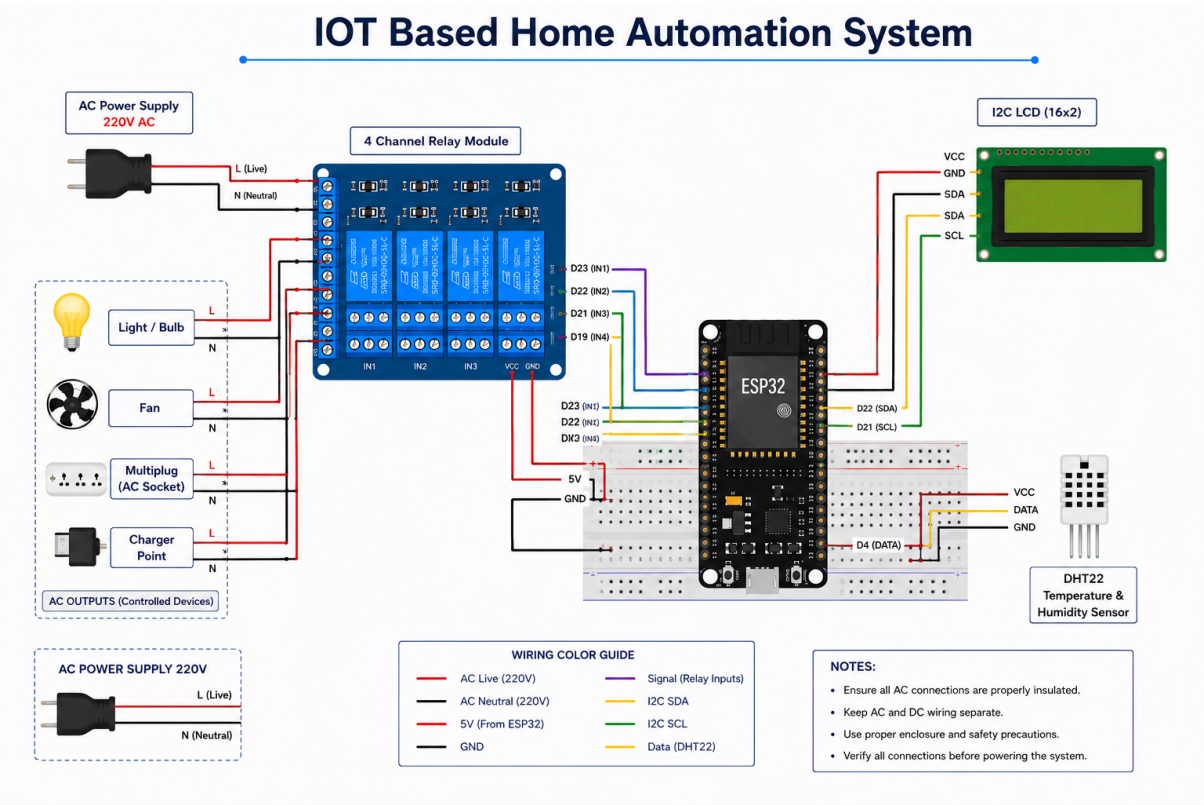


Figure 4.1: Hardware Circuit Diagram – ESP32 with 4-Channel Relay Module, DHT22 Sensor, and I2C LCD

Chapter 5: Project Planning and Scheduling

This chapter explains how we plan to use our time and resources to build the IoT-based Home Automation System. The schedule covers writing the ESP32 firmware in C/C++, developing the web dashboard, wiring the breadboard prototype (ESP32, relay module, and DHT22 sensor), and integrating and testing all the hardware and software parts together.

5.1 Tasks

The following tasks must be completed to deliver a fully working home automation system:

Table 5.1: Project Task List

#	Task Description	Duration	Expected Output	Owner
1	Project Kickoff: Discuss project scope, assign team roles (firmware, UI, hardware, documentation), agree on coding standards	Week 1	role assignment sheet, coding guidelines	All
2	Literature Review: Study ESP32 Arduino-core docs, web server API in C++, DHT22 datasheet, HTML/CSS basics for the dashboard	Week 1–2	Chapter 2 draft, reference list, SDK notes	All
3	System Architecture & Circuit Design: Finalize GPIO-to-relay channel mapping, DHT22 data pin assignment, draw circuit diagram, define dashboard endpoint scheme	Week 2–4	Circuit diagram, URL routing table, pin assignment table	Team

Table 5.1 continued

#	Task Description	Duration	Expected Output	Owner
4	Hardware Procurement & Assembly: Purchase ESP32, 4-channel relay module, DHT22, power supply, wires; assemble circuit on breadboard; verify GPIO-to-relay connection	Week 3–5	Assembled hardware prototype with verified relay switching	Hardware lead
5	Firmware – Wi-Fi & Web Server: Initialize ESP32 in STA mode; connect to Wi-Fi; start the web server; implement the dashboard route handlers	Week 5–7	ESP32 connects to Wi-Fi, responds to browser requests from the dashboard	Firmware lead
6	Firmware – GPIO Relay Control: Write functions to toggle each relay channel from dashboard button clicks; test each relay physically	Week 6–8	All 4 relay channels switch correctly from browser button click	Firmware lead
7	Firmware – DHT22 Sensor: Integrate the DHT22 driver; read temperature and humidity; add automatic fan-control threshold logic	Week 7–9	Live temperature & humidity values readable via serial monitor	Firmware lead
8	Dashboard UI Design: Design the dashboard in HTML/CSS (relay buttons, sensor display, status colours); serve it from the ESP32 web server	Week 7–10	Compact HTML/CSS C string serving complete dashboard in browser	UI lead
9	Dashboard Data Integration: Update the dashboard with live relay states and sensor values on every page load/refresh	Week 9–11	Browser shows correct relay states (green/red) and live sensor data	Firmware + UI

Table 5.1 continued

#	Task Description	Duration	Expected Output	Owner
10	End-to-End Integration & Testing: Connect all hardware; flash complete firmware; test all use cases from the browser (toggle all relays, automatic fan control, sensor view)	Week 11–12	Fully tested, working system accessible from phone and PC browsers	All
11	Debugging & Optimization: Fix any firmware issues or incorrect relay states; verify stable Wi-Fi and dashboard response under continuous use	Week 12–13	Stable, optimized system with no known bugs	Firmware lead
12	Documentation: Write all report chapters, produce final figures, tables, ER diagram, Gantt chart; peer-review each section	Week 11–14	Complete, reviewed project report draft	All
13	Final Demo, Supervisor Review & Submission: Present live system demonstration; incorporate supervisor feedback; submit report and source code	Week 14	Submitted report + source code + working demo	All
14	Future Enhancement: Prototype the INMP441/MAX98357A voice-feedback module on the breadboard and test basic status announcements	Post-submission	Working voice-feedback proof of concept	All

5.2 Gantt Chart

The Gantt chart below illustrates the complete project schedule across 14 weeks. Shaded cells indicate the active period for each task. Firmware development for the web server,

GPIO relay control, and DHT22 sensor integration overlaps and runs across Weeks 5–9 to keep the schedule efficient, while dashboard UI work runs in parallel from Week 7. *(Note: the task list here has been corrected to match this project – the original draft's Gantt chart referenced unrelated tasks such as "Socket Communication Implementation".)*

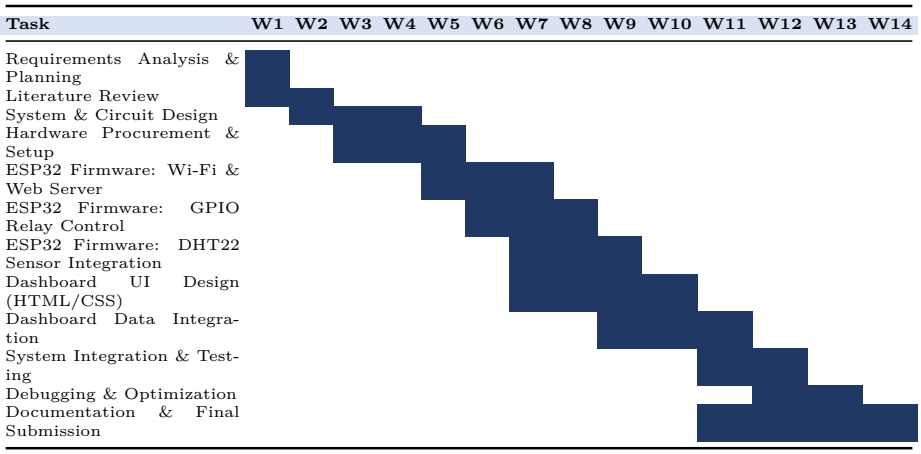


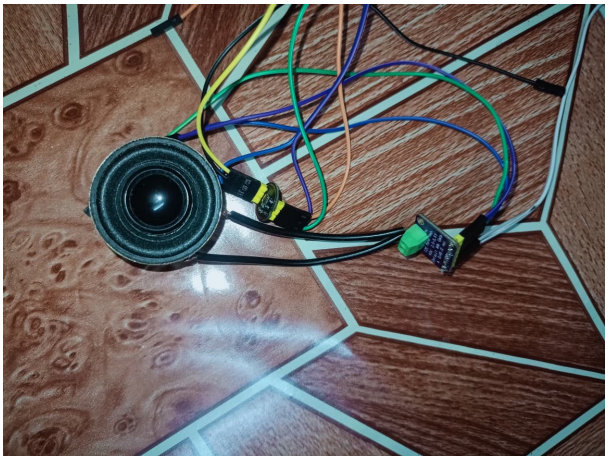
Figure 5.1: Gantt Chart – Project Schedule (14 Weeks)

5.3 Conclusion and Future Enhancements

This project set out to design and build a self-hosted, affordable home automation system around the ESP32 microcontroller, and the working breadboard prototype demonstrates that this goal has been met. The system allows four household appliances to be controlled remotely through a custom web dashboard, uses a DHT22 sensor to automatically manage a fan based on real temperature and humidity readings, and does all of this without relying on any third-party cloud service or mobile app. Testing of the individual components and the integrated prototype confirmed that the relay switching, sensor readings, Wi-Fi connectivity, and dashboard all function reliably together, meeting the objectives set out in Chapter 1.

As a planned extension to the current system, we intend to add a voice-feedback module that lets the system speak out the status of each appliance instead of only showing it on the web dashboard. The module is built around an INMP441 I2S MEMS microphone, which will capture short voice commands, and a MAX98357A I2S Class-D amplifier driving a small 3W, 4Ω speaker, which will play back short confirmation messages.

Whenever a relay is toggled, either from the dashboard or by a spoken command, the ESP32 will announce the change through the speaker, for example saying "Fan turned on" or "Light turned off," and can later be extended to read out the live temperature and humidity from the DHT22 sensor on request. Because the INMP441 and MAX98357A both use the I2S digital audio protocol, they connect directly to the ESP32's I2S peripheral without needing a separate DAC or analog front end, which keeps the extra wiring



on the breadboard prototype simple.

The additional components planned for this enhancement are:

- INMP441 I2S MEMS microphone – captures spoken commands through the I2S digital audio interface
- MAX98357A I2S Class-D amplifier – decodes the I2S audio stream and drives the speaker directly
- 3W, 4Ω speaker – plays back spoken confirmations and status announcements

Early wiring of this voice-feedback module has already begun on a separate breadboard, connecting the small speaker to the MAX98357A amplifier and the INMP441 microphone, ahead of its integration with the main ESP32 firmware.

This enhancement is intended to make the system more accessible, for example for users who prefer audio confirmation over reading a screen, and is planned as the next phase of development after the current relay-and-sensor prototype is finalized.

Bibliography

[1] Adafruit Industries. (n.d.-a). *DHT11, DHT22 and AM2302 sensors*. Adafruit Learning System. https://learn.adafruit.com/dht

[2] Adafruit Industries. (n.d.-b). *Adafruit MAX98357 I2S Class-D mono amp*. Adafruit Learning System. https://learn.adafruit.com/adafruit-max98357-i2s-class-d-mono-amp

[3] Anusha, K., Aravind, S., Umamaheshwar Rao, N., & Ravisagar, T. (2024). Internet of Things (IoT) based home automation using ESP-32. *International Journal for Research in Applied Science and Engineering Technology*. https://doi.org/10.22214/ijraset.2024.61912

[4] Espressif Systems. (n.d.-a). *ESP32 series technical reference manual*. https://www.espressif.com/sites/default/files/documentation/esp32_technical_reference_manual_en.pdf

[5] Espressif Systems. (n.d.-b). *Arduino core for ESP32 documentation*. https://docs.espressif.com/projects/arduino-esp32/

[6] InvenSense Inc. (n.d.). *INMP441 omnidirectional microphone with bottom port and I^2S digital output* [Datasheet]. https://www.invensense.com/wp-content/uploads/2015/02/INMP441.pdf

[7] Kakani, D., Kulkarni, A., Pandya, Y., & Mehendale, N. (2024). IoT-based smart home automation using ESP32 and Blynk with enhanced reliability. *SSRN*. https://papers.ssrn.com/sol3/papers.cfm?abstract_id=4802741